
Kubernetes Security (Expanded Overview)

1 Secret

A built-in Kubernetes resource to **store sensitive data** such as passwords, OAuth tokens, and SSH keys.

◆ Key Features:

- Stored as **base64-encoded** values (not encrypted by default).
- Mounted as environment variables or volumes.
- Can be used by Pods without hardcoding sensitive info in manifests.

Example:

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  username: YWRtaW4= # "admin"
  password: c2VjcmV0 # "secret"
```

2 ConfigMap

Stores **non-sensitive configuration** in key-value format. Used to decouple app configuration from container image.

◆ Key Features:

- Mounted into Pods as volumes or environment variables.
- Useful for runtime configs, URLs, flags, etc.

Example:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_MODE: production
  TIMEOUT: "30"
```

3 PodSecurityContext

Sets **security-related parameters** for Pods or containers, such as user IDs, SELinux, capabilities, and filesystem options.

◆ Key Features:

- Can enforce running containers as non-root.
- Supports fine-grained security settings at pod/container level.

Example:

```
spec:
  securityContext:
    runAsUser: 1000
    runAsNonRoot: true
    fsGroup: 2000
```

4 Pod Security Admission (PSA)

A **built-in admission controller** that enforces Kubernetes **Pod Security Standards** at the namespace level.

◆ Security Profiles:

- **Privileged** – No restrictions (unsafe for most use cases).
- **Baseline** – Prevents known privilege escalations.
- **Restricted** – Enforces strictest settings (non-root, no privilege escalation).

Example:

```
kubectl label namespace dev pod-security.kubernetes.io/enforce=restricted
```

This enforces the "restricted" policy on the `dev` namespace.

5 RBAC (Role-Based Access Control)

Controls **who can perform what actions** on which Kubernetes resources.

◆ Key Features:

- Enforced by the **kube-apiserver**.
- Uses `Role`, `ClusterRole`, `RoleBinding`, `ClusterRoleBinding`.

Example:

```
apiVersion: rbac.authorization.k8s.io/v1
```

```
kind: Role
```

```
metadata:
```

```
  namespace: dev
```

```
  name: pod-reader
```

```
rules:
```

```
- apiGroups: [""]
```

```
  resources: ["pods"]
```

```
  verbs: ["get", "watch", "list"]
```

6 Role / ClusterRole

Define a **set of permissions** (verbs + resources).

- **Role** – Namespace-scoped.
 - **ClusterRole** – Cluster-wide or reusable in any namespace.
-

7 RoleBinding / ClusterRoleBinding

Grants access defined in a **Role** or **ClusterRole** to a **user, group, or ServiceAccount**.

- **RoleBinding** – Applies to one namespace.
- **ClusterRoleBinding** – Applies cluster-wide.

Example:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: dev
subjects:
  - kind: User
    name: alice
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

8 Security Policies (OPA / Gatekeeper)

Enforce **custom and complex security rules** using the **Open Policy Agent (OPA)** and **Gatekeeper**.

◆ **Key Features:**

- Define policies in **Rego** (policy language).
- Blocks deployments that violate rules (e.g., no **latest** tag, no privileged Pods).

Example:

- Deny Pods using the **latest** image tag:

```
violation[{"msg": msg}] {
  input.review.object.spec.containers[_].image == "nginx:latest"
  msg := "Using 'latest' image tag is not allowed"
}
```

9 NetworkPolicy Controller

Enforces **NetworkPolicies** to control traffic **between Pods** or from **external sources**.

◆ Key Features:

- Enforced by the **CNI plugin** (e.g., Calico).
- Restricts ingress/egress traffic based on labels, IPs, ports.

Example:

Allow traffic only from `app=frontend` Pods:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend
spec:
  podSelector:
    matchLabels:
      role: backend
  ingress:
    - from:
      - podSelector:
          matchLabels:
            app: frontend
```



Summary Table

	Component	Description	Scope/Use Case
1	Secret	Stores sensitive data (base64-encoded)	Passwords, tokens, certificates
2	ConfigMap	Key-value config for non-sensitive data	App configs, URLs, runtime variables

3	PodSecurityContext	Sets security settings at Pod/Container level	Run as non-root, set user/group IDs
4	Pod Security Admission	Namespace-level pod security enforcement	Restricted, Baseline, Privileged modes
5	RBAC	Manages access to Kubernetes resources	Who can do what on which resources
6	Role / ClusterRole	Define specific permissions	Read-only, admin, pod access, etc.
7	RoleBinding / ClusterRoleBinding	Assigns roles to users or service accounts	Grant access to users/groups
8	OPA / Gatekeeper	Policy engine for complex, custom rules	Prevent bad configurations
9	NetworkPolicy Controller	Enforces traffic rules between Pods	Isolate or allow communication based on labels/IPs
